

PYTHON

Module 27: Temporal Data & Datetime

CODING

Temporal Data: Workings with Dates

In Python, dates are not a primitive data type. Instead, we utilize the built-in `datetime` module to manipulate temporal data objects. At CodeHive, mastering time is essential for logging, scheduling, and data analysis.

- ****The datetime object****: A single object containing year, month, day, hour, minute, second, and microsecond data.

Acquiring Current Time

To capture the exact moment a script runs, use the `now()` method.

- **Syntax**: `datetime.datetime.now()`
-

• **Extraction**: Once captured, you can access specific attributes like `.year`, `.month`, or `.day` directly from the resulting object.

Constructing Specific Dates

You can create a custom date object using the `datetime()` constructor.

- ****Required Arguments****: Year, Month, Day.

- ****Optional Arguments****: Hour, Minute, Second, Microsecond, and Timezone (tzzone).

Example: `datetime.datetime(2020, 5, 17)` perfectly captures May 17th, 2020.

Formatting: The strftime()

Method

Raw date objects are often not user-friendly. The `strftime()` (string format time) method converts date objects into readable strings using specific directives.

- **`**%A**`**: Full weekday name (e.g., Wednesday).

- **`**%B**`**: Full month name (e.g., December).

- **`**%Y**`**: Full four-digit year (e.g., 2026).

- **`**%d**`**: Day of the month (01-31).

Directives Table for Developers

- `%H/%I` : Hour (24h vs 12h format).

- `%M / %S` : Minutes and Seconds.

- `%p` : Locale equivalent of AM or PM.

- `%j` : Day of the year (001-366).

- `%c` : Local version of date and time (comprehensive string).

CodeHive Standard for Timezones

By default, datetime objects are 'naive' (lack timezone info). For international applications, always consider attaching a timezone object to ensure consistency across geographically distributed systems.

CODEHIVE